MEMBER 5 HANDBOOK

FLOW OF WORK + DELIVERABLES

(MEMBER 5 — ML MODEL TRAINING ENGINEER)

Primary responsibility:
Develop, train/fine-tune, evaluate, and package the machine-learning model that can determine whether two medical invoice images are based on the **same or highly similar document template**, and provide the trained model + inference specifications to Member 3 for integration into the AI microservice.

PAGE 1 — WHAT THIS HANDBOOK ANSWERS

Questions answered in this handbook

1. What exactly is Member 5 responsible for?
2. What does "training the model" actually mean in this project?
3. What exactly am I receiving from Member 4?
4. What is an embedding?
5. Why do we need embeddings?
6. What is the difference between a pretrained model and our trained model?
7. Do I need to train a model from scratch?
8. Which programming language should I use?
9. Which ML framework should I use?
10. Where should I find/select the pretrained model?
11. Which type of model should I start with?
12. What is the recommended baseline approach?

13. What is a Siamese network?
14. What is contrastive learning?
15. What exactly is being learned by the model?
16. What does pairs.csv contain?
17. How does train.csv enter the training process?
18. What happens during one training iteration?
19. What does the model output during training?
20. How are embeddings generated?
21. What is cosine similarity?
22. How do we decide whether two invoices have the same template?
23. What is a similarity threshold?
24. How should the model be evaluated?
25. What are training, validation and test datasets?
26. Why must template families be separated between splits?
27. What is data leakage?
28. What metrics should be reported?
29. What hardware is required?
30. Where should training be performed?
31. What exactly should be saved after training?
32. What does Member 5 hand over to Member 3?
33. What should Member 5 NOT do?
34. What is the complete ML workflow?
35. What defines "DONE" for Member 5?

---

1. What exactly is Member 5 responsible for?

Member 5 owns the **ML model development lifecycle**.

Your workflow starts here:

Member 4

↓

Training dataset

↓

Member 5

↓

Pretrained model

↓

Baseline experiment

↓

Fine-tuning

↓

Evaluation

↓

Trained model

↓

Member 3

You are responsible for answering:

Can our model reliably identify whether two invoice documents have the same underlying template despite changes to patient information, provider information, amounts, dates, logos, etc.?

---

2. What does "training the model" mean here?

You are **not** teaching a computer what an invoice is from zero.

You are starting with a model that has already learned useful visual/document representations from a large dataset.

Then you teach it:

"For our specific problem, these two documents should be considered similar, while these two should be considered different."

For example:

Invoice A + Invoice B

↓

SAME

↓

Label 1

and:

Invoice A + Invoice C

↓

DIFFERENT

↓

Label 0

The model gradually adjusts its internal parameters so that:

same template → similar representation

different template → different representation

---

3. What exactly will Member 4 give you?

You should receive:

dataset/

|

├── raw/

├── generated/

│

└── metadata/

    ├── documents.csv

    ├── pairs.csv

    ├── train.csv

    ├── validation.csv

    └── test.csv

Your most important files are:

train.csv

validation.csv

test.csv

---

4. What does train.csv actually contain?

For example:

document_a,document_b,label

DOC001,DOC003,1

DOC001,DOC004,1

DOC001,DOC018,0

DOC003,DOC004,1

DOC003,DOC021,0

Where:

1 = same/similar template

0 = different template

The actual images are stored separately.

The CSV simply tells your training program:

"Here are two images. The correct relationship between them is 1."

---

5. What is an embedding?

This is one of the most important concepts for this project.

An **embedding is a numerical representation of a document**.

Imagine:

invoice.jpg

↓

ML model

↓

[0.14, -0.37, 0.82, 0.21, ...]

That list of numbers is the document's **embedding vector**.

Instead of comparing two entire images pixel-by-pixel, we compare their numerical representations.

For example:

Invoice A

↓

Vector A

Invoice B

↓

Vector B

Then:

Vector A ↔ Vector B

can be compared mathematically.

---

6. Why do we need embeddings?

Because our actual question is:

"How similar are these two document templates?"

We want something like:

Invoice A

↓

Embedding A

Invoice B

↓

Embedding B

Embedding A

+

Embedding B

↓

Similarity = 0.94

while:

Invoice A

+

Completely different invoice

↓

Similarity = 0.28

The closer the representations are, the more likely they belong to the same template family.

---

7. Do you train a model from scratch?

No.

Do not train a deep-learning model from random initialization.

You have only 12 days and a relatively small dataset.

Instead:

Large pretrained model

↓

Your invoice dataset

↓

Fine-tuning

↓

Your specialized model

This is called **transfer learning/fine-tuning**.

---

8. What programming language should you use?

Use:

Python

This is the ML side of the project.

Recommended stack:

Python

PyTorch

Hugging Face Transformers

Hugging Face Datasets

Pillow

OpenCV

scikit-learn

NumPy

Matplotlib

You do **not** need TensorFlow unless the person already knows it.

For this project I'd standardize on:

Python + PyTorch + Hugging Face

---

9. Where do you find the pretrained model?

Use the **Hugging Face Model Hub**.

The model-training engineer should search for pretrained vision/document models there.

Possible model families to investigate include:

- DINOv2
- CLIP
- LayoutLMv3
- other suitable document/vision transformers

However:

Do not blindly choose LayoutLMv3 simply because it sounds like a document model.

Your first objective is to find which pretrained representation produces useful similarity scores for **invoice templates**.

---

10. What model should you start with?

For your 12-day deadline, I recommend a two-stage approach.

Stage 1 — Baseline

Start with a pretrained visual/document representation model.

A strong candidate to test first is:

DINOv2

Why?

It provides strong visual representations without requiring you to build a complicated document-understanding system first.

You can also test CLIP as a comparison.

The baseline question is:

Without any fine-tuning, do embeddings from this model already separate similar invoice templates from different templates?

---

11. What does the baseline experiment look like?

Take:

DOC001

DOC002

DOC003

...

Generate:

DOC001 → embedding

DOC002 → embedding

DOC003 → embedding

Then use cosine similarity.

Suppose:

Same template:

DOC001 ↔ DOC002 = 0.93

DOC001 ↔ DOC003 = 0.91

DOC002 ↔ DOC003 = 0.95

Different templates:

DOC001 ↔ DOC020 = 0.31

DOC001 ↔ DOC027 = 0.42

DOC002 ↔ DOC030 = 0.28

That's promising.

---

12. What if the baseline isn't good?

Then fine-tune.

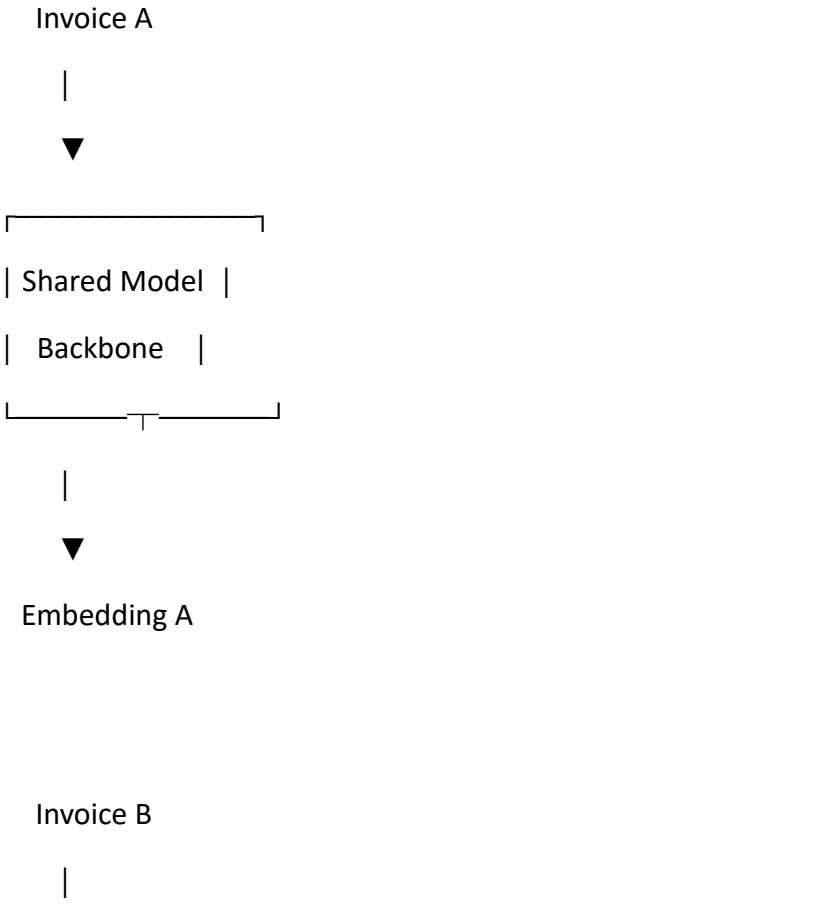
This is where your dataset becomes valuable.

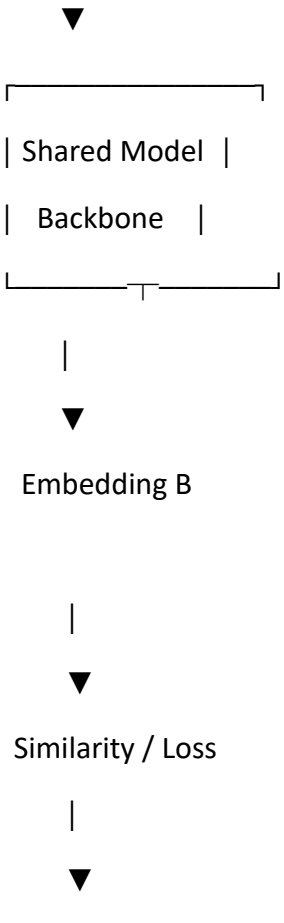
---

13. Recommended training architecture

I would use a **Siamese/metric-learning architecture**.

Conceptually:





The **same model weights** are used for both images.

That's why it's called a Siamese architecture.

---

14. What is the model actually learning?

Suppose:

Invoice A

+

Synthetic variation of A

=

Label 1

The model learns:

These two should have similar embeddings.

But:

Invoice A

+

Completely different invoice

=

Label 0

The model learns:

These should have different embeddings.

Over many examples, the model learns representations that emphasize useful template characteristics.

---

15. What is contrastive learning?

Contrastive learning essentially teaches:

Pull similar examples together and push dissimilar examples apart in representation space.

Imagine:

BEFORE TRAINING

A • B •

C •

AFTER TRAINING

A • B •

A'• B'•

C •

D •

Same-template documents become closer.

Different-template documents become farther apart.

---

16. What does one training iteration actually look like?

Suppose train.csv gives:

DOC001

DOC002

1

Your training code:

Step 1

Load:

DOC001.jpg

DOC002.jpg

Step 2

Preprocess them.

Step 3

Pass both through the shared model.

DOC001 → embedding A

DOC002 → embedding B

Step 4

Calculate similarity/distance.

Step 5

Compare prediction with the known label:

Actual = 1

Predicted = 0.72

Step 6

Calculate loss.

Step 7

Backpropagate.

Step 8

Update model weights.

Then repeat for the next pair.

This happens thousands of times during training.

You do not manually correct the model.

The optimization algorithm does it.

---

17. What is validation.csv used for?

It is **not used to train the model's weights**.

You use it to determine things such as:

- Which model performs better?
- Which hyperparameters work better?
- Which similarity threshold is appropriate?
- When should training stop?

For example:

Threshold 0.70 → F1 = 0.78

Threshold 0.75 → F1 = 0.83

Threshold 0.80 → F1 = 0.89

Threshold 0.85 → F1 = 0.86

You would probably select something around 0.80 based on validation performance.

The exact threshold should come from your experiments.

---

18. What is test.csv used for?

Only final evaluation.

After you've finished selecting your model and threshold:

Final model

↓

test.csv

↓

Final metrics

Don't repeatedly tune your model against the test set.

Otherwise the test set stops being a genuine measure of generalization.

---

19. Very important: avoid data leakage

Suppose:

T001 original

T001 synthetic 1

T001 synthetic 2

T001 synthetic 3

You cannot blindly do:

TRAIN:

T001 original

TEST:

T001 synthetic 2

because the model has effectively seen the same underlying template during training.

Instead, whenever possible, split by **template family**.

For example:

TRAIN

T001

T002

T003

T004

T005

T006

VALIDATION

T007

T008

TEST

T009

T010

This provides a much more convincing evaluation:

Can our model recognize template similarity involving template families it hasn't directly trained on?

---

20. What should the model ultimately output?

The most useful final model output is an **embedding**.

For example:

invoice.jpg

↓

trained model

↓

embedding

Something like:

[0.12, -0.41, 0.82, ...]

The model doesn't necessarily need to directly output:

FRAUD

Instead:

Model

↓

Embedding

↓

Cosine similarity

↓

Similarity threshold

↓

Potential template match

Then Member 3's AI microservice can build the fraud-risk logic around this.

---

21. What is cosine similarity?

It's a mathematical measure of how similar two vectors point in the same direction.

For your purposes:

~1.0 → very similar

~0.0 → very different

In practice, don't assume a universal threshold like 0.80.

Your job is to **experimentally determine the threshold** using validation data.

---

22. Example of your final system

Suppose Member 3 receives a new invoice.

Your trained model generates:

New invoice

↓

Embedding A

Existing invoice:

Claim 182

↓

Embedding B

Then:

cosine(A, B) = 0.94

Your model-training deliverable should tell Member 3:

"Based on validation experiments, a similarity around X is the appropriate operating threshold under our evaluation setup."

Member 3 can then use that threshold in the AI service.

---

23. What metrics should you report?

At minimum:

Precision

Of documents flagged as similar, how many actually were similar?

Recall

Of genuinely similar documents, how many did we detect?

F1-score

Balance between precision and recall.

Accuracy

Overall correct predictions.

Confusion matrix

Show:

Predicted

Similar Different

Actual Similar

Actual Different

Also produce:

Similarity distribution

Show the distribution of:

Positive pairs

vs

Negative pairs

This is extremely useful for your SIH presentation.

---

24. What should a good result look like?

Ideally:

Similarity

0.0 0.5 1.0

|----------|----------|

Different templates

[redacted]

Same templates

[redacted]

There should be a reasonably clear separation.

If the distributions heavily overlap:

Different:

[redacted]

Same:

[redacted]

then the model isn't distinguishing templates well enough.

That needs to be addressed before moving on.

---

25. What tools should Member 5 actually use?

Development

VS Code / PyCharm / Jupyter Notebook

Any is fine.

ML framework

PyTorch

Model library

Hugging Face Transformers

Dataset handling

Hugging Face Datasets / Pandas

Image handling

Pillow / OpenCV

Metrics

scikit-learn

Visualization

Matplotlib

Experimentation

Jupyter notebooks are useful during the POC.

Once the approach is finalized, convert the important work into reproducible Python scripts.

---

26. Where should training happen?

Do **not** depend on your Dell Latitude for serious fine-tuning.

Use a GPU environment.

Possible choices:

- Google Colab
- Kaggle notebooks
- university GPU
- another available cloud GPU

The workflow:

GitHub repository

↓

Clone

↓

Dataset

↓

GPU environment

↓

Train

↓

Evaluate

↓

Export model

---

27. What hardware does Member 5 need?

The important hardware is:

GPU

A CUDA-compatible NVIDIA GPU is ideal.

The exact GPU depends on the model, image size, batch size and whether you're using mixed precision.

For your 12-day project, **use a cloud GPU rather than wasting time trying to train on a CPU laptop.**

---

28. What files should Member 5 create?

Use:

ml-training/

|

├── notebooks/

| ├── 01_baseline_embeddings.ipynb

| ├── 02_similarity_analysis.ipynb

| └── 03_training_experiments.ipynb

|

├── scripts/

| ├── generate_embeddings.py

| ├── train.py

| └── evaluate.py

|

├── configs/

| └── training_config.yaml

|

├── outputs/

| ├── baseline/

| ├── experiments/

| ├── metrics/

| └── trained_model/

|

├── requirements.txt

└── README.md

---

29. What should generate_embeddings.py do?

It should be able to:

Input:

invoice image

Output:

embedding vector

This is useful both for experiments and for understanding exactly what Member 3 will eventually need during inference.

---

30. What should train.py do?

Conceptually:

Load training dataset

↓

Load pretrained model

↓

Create Siamese/metric-learning architecture

↓

Load image pairs

↓

Train

↓

Validate after epochs

↓

Save best model

It should accept configuration rather than having everything hardcoded.

---

31. What should evaluate.py do?

It should:

Load trained model

↓

Load test.csv

↓

Generate embeddings

↓

Calculate similarities

↓

Apply threshold

↓

Calculate metrics

↓

Save results

Output something like:

```
{
  "accuracy": 0.91,
  "precision": 0.89,
  "recall": 0.94,
  "f1": 0.91,
  "threshold": 0.81
}
```

---

32. What exactly must Member 5 hand over to YOU?

This is extremely important because **you are Member 3 and will build the AI microservice.**

Member 5 must give you:

1. Trained model

trained_model/

2. Model architecture information

You need to know:

Which backbone?

Which processor?

Input image dimensions?

Embedding dimension?

Normalization?

---

3. Preprocessing instructions

For example:

Resize → 224×224

Normalize → X

RGB → yes

You must reproduce this **exactly** in your AI microservice.

---

4. Embedding specification

For example:

Embedding dimension = 768

---

5. Similarity method

Cosine similarity

---

6. Recommended threshold

For example:

Threshold = 0.81

along with validation evidence explaining how it was selected.

---

7. Evaluation report

For example:

Accuracy: 91%

Precision: 89%

Recall: 94%

F1: 91%

---

8. Inference script

This is extremely useful.

Member 5 should provide:

predict.py

which proves:

invoice.jpg

↓

trained model

↓

embedding

You can use this as the reference implementation when creating your FastAPI service.

---

33. What should NOT be handed over?

Don't just send Member 3:

model.pth

and say:

"Here is the model."

That's insufficient.

Member 3 needs to know **exactly how the model expects an image to be processed**.

A model is only useful if the inference pipeline reproduces its training conditions.

---

34. What should Member 5 NOT do?

Member 5 should **not**:

- Build FastAPI
- Build Spring Boot
- Build React

- Implement the final fraud dashboard
- Create frontend APIs
- Train OCR from scratch
- Manually alter model predictions
- Randomly choose a similarity threshold
- Claim similarity equals confirmed fraud

The model's job is primarily:

Learn a useful representation of invoice templates.

The application layer handles the rest.

---

35. Final folder structure

At completion:

```
ml-training/
|
├── notebooks/
|   ├── 01_baseline_embeddings.ipynb
|   ├── 02_similarity_analysis.ipynb
|   └── 03_training_experiments.ipynb
|
├── scripts/
|   ├── generate_embeddings.py
|   ├── train.py
|   └── evaluate.py
|
├── configs/
|   └── training_config.yaml
```

```
|
├── outputs/
|   ├── baseline/
|   |   ├── similarity_results.csv
|   |   └── similarity_distribution.png
|   |
|   ├── metrics/
|   |   ├── metrics.json
|   |   └── confusion_matrix.png
|   |
|   └── trained_model/
|       └── [model files]
|
├── requirements.txt
└── README.md
```

If the trained model is too large for normal GitHub storage, **do not force it into the repository**. Put the model artifact in an appropriate model/object-storage location and document the download path in README.md.

---

36. What should README.md contain?

At minimum:

1. Objective
2. Dataset used
3. Model selected
4. Why the model was selected
5. Input preprocessing

6. Training methodology

7. Loss function

8. Hyperparameters

9. Training environment

10. Evaluation methodology

11. Final metrics

12. Similarity threshold

13. How to load the model

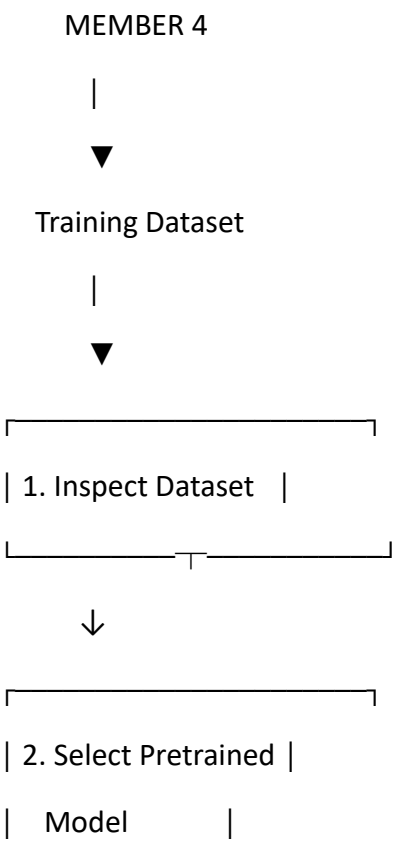
14. How to generate an embedding

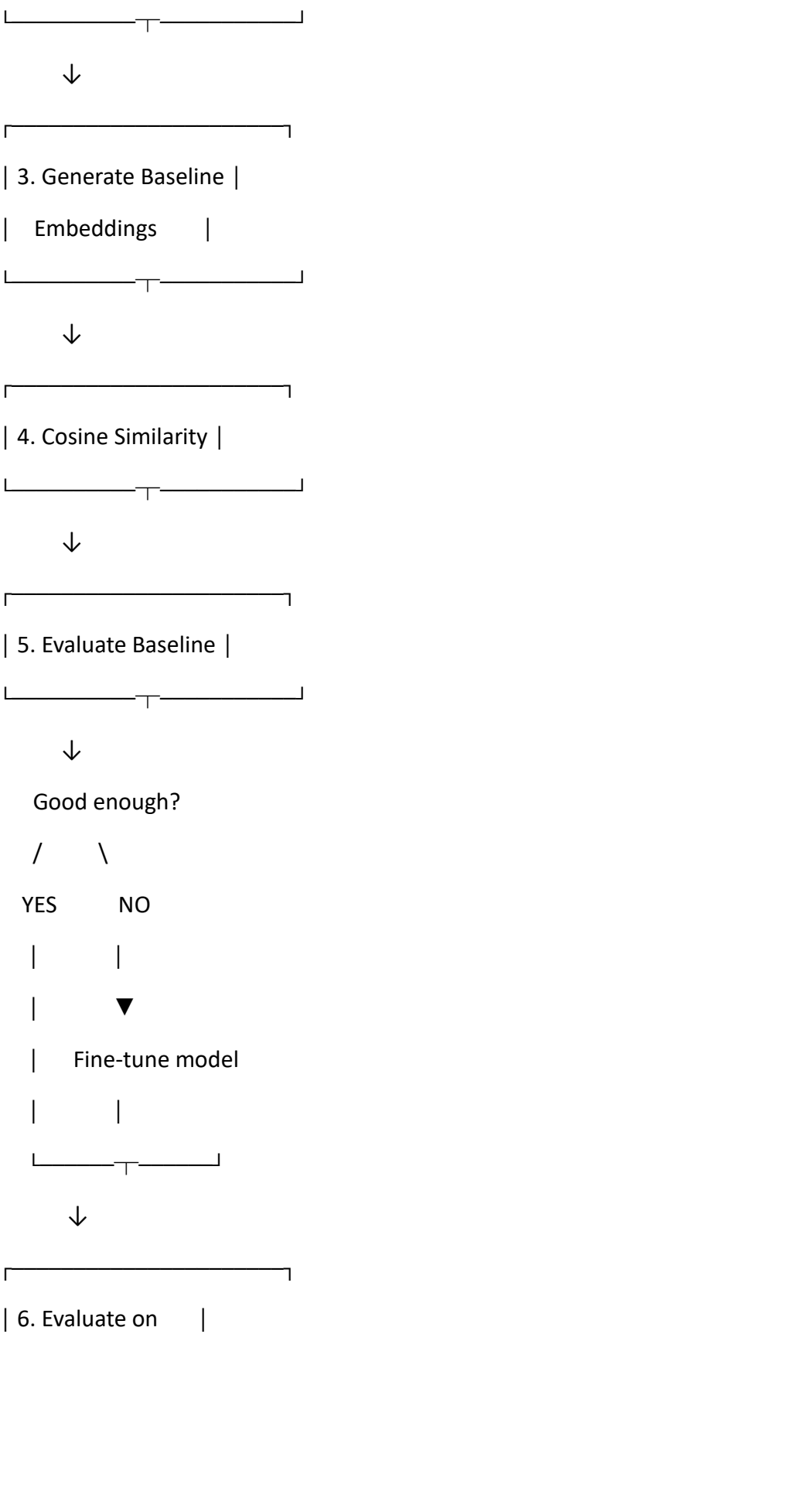
15. Example inference

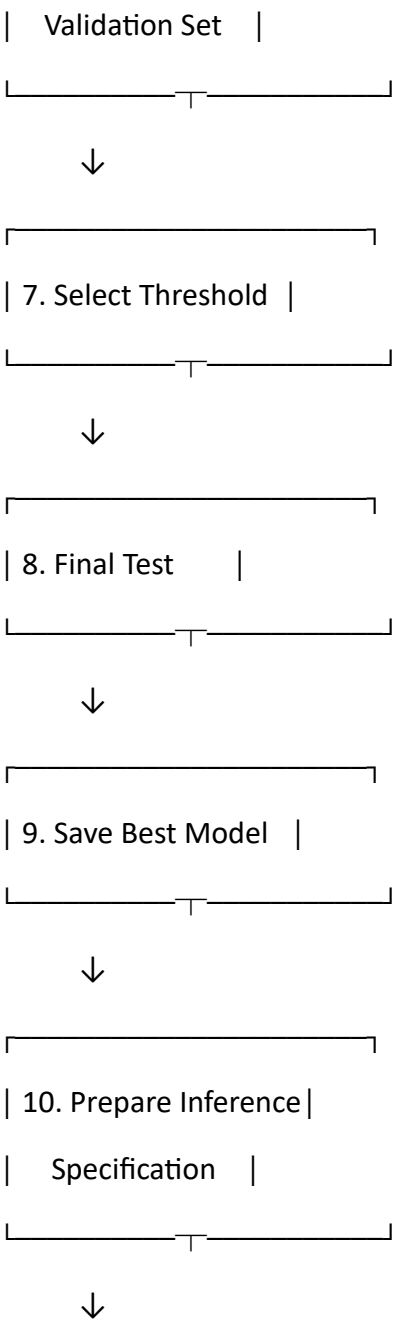
16. Known limitations

---

37. Complete Member 5 workflow




3. Generate Baseline Embeddings
4. Cosine Similarity
5. Evaluate Baseline
Good enough?
YES
NO
Fine-tune model
6. Evaluate on



---

38. Definition of "DONE" for Member 5

Member 5 is **DONE** only when all of the following are available:

Dataset

train.csv

validation.csv

test.csv

Model

A trained/fine-tuned model that can generate invoice embeddings.

Evaluation

At minimum:

Precision

Recall

F1

Accuracy

Confusion Matrix

Similarity Distribution

Threshold

A threshold selected using validation data and documented.

Reproducibility

A script that can load the model and generate an embedding for a new invoice.

Documentation

A clear model report.

Handoff to Member 3

Member 3 receives:

Trained model

+

Preprocessing requirements

+

Embedding dimensions

+

Inference code

+

Similarity method

+

Threshold

+

Evaluation metrics

The single most important thing Member 5 should remember

Do not spend the first three days trying to fine-tune a fancy model.

The correct order is:

Pretrained model → baseline embeddings → cosine similarity → evaluate → only then fine-tune if necessary.

Your 12-day deadline makes this especially important. If the pretrained representation already separates your invoice templates reasonably well, you can spend your remaining time making the system robust instead of burning days training a model unnecessarily.

And the ultimate handoff to you is very simple:

MEMBER 5

|

|

trained model

|

preprocessing rules

|

embedding generator

|

similarity threshold

|

▼

MEMBER 3

|

▼

AI MICROservice

That is the boundary between **ML development** and **ML inference engineering** in your project.